

# Design choice justification

We had to implement two metamodels in Ecore: a High-Level Language (HLL) metamodel, and a Pivot Language metamodel. This document justifies the key decisions made during their design.

## High-level Language Metamodel

- **Root Structure: Story → Chapter → Scene**

The HLL root is **Story**, which aggregates **Chapters** and, through them, **Scenes**. This three-level containment hierarchy reflects the authoring vocabulary of Life is Strange (five episodes, each with multiple scenes).

- **Scene: isStart / isEnding Flags vs. Dedicated Subclasses**

Scenes use boolean flags **isStart** and **isEnding** rather than specialised subclasses. This keeps the metamodel flat and makes instance models simpler to write and to read. The EVL constraints (exactly one start, no outgoing choices on endings) impose the required semantics without requiring the extra classes.

Ending scenes additionally carry an optional **endingType:EndingType** enumeration (GOOD/BAD/NEUTRAL), which the authors can classify the narrative outcomes.

- **Choice, Condition, Effect, and DiceRule**

The **Choice** class bundles the label, the target scene reference, an optional **Condition**, an optional **DiceRule**, and zero or more **Effects**. This design keeps all transition-related information in one place, reducing the cognitive load on authors.

The **Condition** class stores the expression as a plain **EString** together with cross-references to the **Variables** and **InventoryItems** the expression mentions. This dual representation is intentional: the expression field is what the author reads and the reference fields allow EVL to validate that referenced entities are declared without parsing the expression string.

The **DiceRule** on a Choice represents a probabilistic branching that the author designs. It carries its own **onSuccess** and **onFailure** scene references, separate from the Choice's normal **target**. This is the only HLL concept that already exposes two successor states explicitly.

- **Variables, Inventory, and Characters at Story Scope**

All **Variables**, **InventoryItems**, and **Characters** are declared at **Story** scope rather than at scene scope. This mirrors how Life is Strange works: **rv\_key** is a global state that persists across the entire story.

# Pivot Language Metamodel

- **Flattening to NarrativeGraph (Node + Edge)**

The pivot completely removes the Story → Chapter → Scene hierarchy. It presents the story as a flat **NarrativeGraph** containing **Nodes** and **Edges**.

Chapter information is deliberately discarded in the pivot because it carries no operational semantics.

- **Node: Merging Scene into a Computational State**

Each HLL **Scene** becomes a **Node** in the pivot. The Node retains only the operationally relevant attributes: **id**, **label**, **isStart**, **isAccepting** (the pivot term for **isEnding**), and **endingType**. Description text, character associations, and other authoring metadata are dropped.

- **Edge: Unifying Choice, DiceRule, and Effects**

Each HLL **Choice** becomes an **Edge** in the pivot. An Edge aggregates:

- A Guard (renamed from Condition) with expression and variable/item cross-references.
- Zero or more Actions (renamed from Effects) typed by ActionType (SET\_VAR, ADD\_ITEM, REMOVE\_ITEM).
- An optional PivotDiceRule carrying successTarget and failureTarget Node references.

The HLL's **DiceRule.onSuccess/onFailure** scene references are flattened into the Edge's **PivotDiceRule** because in the pivot graph, all successors of a node are represented as outgoing edges, whether deterministic or probabilistic.

## Conclusion

The distinction between the HLL and Pivot metamodels are driven by a strict separation of concerns.

The HLL is designed specifically for the human author. It provides hierarchical and domain-specific concepts that make the modelling of the narrative easier.

In contrast, the Pivot metamodel is designed for the machine. It removes narrative metadata and flattens the hierarchy into a simple formal state-machine structure (Nodes and Edges). This transformation is essential to eliminate unnecessary complexity and prepare the model for formal verification (such as NuSMV) and code generation.

In conclusion, the HLL prioritizes the user experience, while the Pivot language focuses strictly on the computational and operational execution.